



**BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA
INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN**

**PROYECTO FINAL
HIPERCUBO**

**ALGEBRA LINEAL CON APLICACIONES
DR. LUIS ERNESTO VALENCIA SEGURA**

DESARROLLADO POR:

DIEGO ARMANDO SIBAJA NERI

MATRICULA: 202460196

JESÚS Yael ROMANO COUTIÑO

MATRICULA: 202477137

CRISTIAN Yael GARCIA GARCIA

MATRICULA: 202443217

AXEL ADAIR ARRIAGA BLANCO

MATRICULA: 202440060

OTOÑO 2025

Introducción

En Álgebra Lineal hemos trabajado con conceptos como vectores, matrices, transformaciones lineales y cambios de base, normalmente aplicados a situaciones de dos o tres dimensiones. Sin embargo, muchas de estas ideas no están limitadas al espacio físico que percibimos; pueden extenderse de manera natural a dimensiones superiores. Para comprobarlo y relacionar la teoría con una aplicación real, desarrollamos un proyecto donde modelamos y animamos un tesseracto o hipercubo, que es la versión de cuatro dimensiones de un cubo tradicional.

Llevamos a la práctica varios de los conceptos estudiados en la materia, veremos cómo se aplican las transformaciones lineales representadas mediante matrices. Para ello, definimos los dieciséis vértices del tesseracto como vectores en un espacio 4D y generamos sus aristas a partir de combinaciones. Una vez establecidos los puntos, implementamos rotaciones en diferentes planos del espacio de cuatro dimensiones, como XY, XW, YZ y ZW. Cada una de estas rotaciones se calculó de forma independiente utilizando funciones que aplican matrices de transformación, lo cual nos permitió observar cómo un objeto que no existe físicamente en nuestro entorno puede ser creado como representación utilizando conceptos del Álgebra Lineal.

Como parte del desarrollo, también realizamos dos proyecciones consecutivas para poder representar el tesseracto en la pantalla. Primero, proyectamos los puntos de 4D a 3D usando un factor dependiente de la componente w , y posteriormente llevamos las coordenadas 3D a posiciones 2D dentro de la ventana del programa. Este procedimiento nos permitió visualizar una figura que, aunque pertenece a un espacio superior, puede ser entendida gracias a las herramientas del álgebra lineal. De esta manera, conceptos como el espacio vectorial, la combinación lineal, los sistemas de coordenadas y las matrices de rotación se utilizan de forma directa y práctica.

Para implementar todo esto utilizamos el lenguaje C++ y la librería `olcPixelGameEngine`, que facilita el dibujo de líneas en pantalla. En nuestro caso, cada línea representa una arista del tesseracto, y la animación surge al aplicar rotaciones en función del tiempo. El proyecto se desarrolló en Visual Studio, donde estructuramos el código, probamos las transformaciones y verificamos que la proyección fuera estable y visualmente clara.

Objetivo

El objetivo de este trabajo es comprender cómo los conceptos de álgebra lineal pueden aplicarse de forma directa en la construcción y visualización de un tesseracto dentro de un programa en C++. Aunque en clase solemos trabajar con ejemplos en dos o tres dimensiones, este proyecto busca ir más allá y mostrar que las mismas ideas, vectores, rotaciones y proyecciones siguen funcionando aun cuando se extienden a un espacio de cuatro dimensiones. La idea es que, más allá del resultado visual, como estudiantes podamos relacionar cada parte del código con una idea algebraica que ya conocemos.

De igual manera, debemos definir correctamente los puntos que forman un tesseracto en 4D, entendiendo que cada punto no es más que un vector con cuatro componentes y que la estructura completa se obtiene combinando todas las variaciones posibles entre valores positivos y negativos. Una vez establecidos estos puntos, lo siguiente consiste en determinar cómo se conectan entre sí, formando las aristas que le dan forma al tesseracto. Esto permite tener una base sólida antes de aplicar cualquier transformación.

Al implementar rotaciones en diferentes planos de cuatro dimensiones, a diferencia de una rotación común en 2D o 3D, aquí las combinaciones posibles aumentan y es necesario analizar cómo interactúan los ejes entre sí. En el código se utilizaron varias rotaciones encadenadas, lo que permite observar un movimiento más completo y menos predecible del objeto.

A lo largo del proyecto, ayuda a entender por qué ciertos elementos se ven más pequeños, más grandes o aparentemente deformados conforme el objeto rota. De esta manera, se observa cómo funciona una transformación al reducir dimensiones sin perder la estructura lógica del objeto. Al interpretar el comportamiento visual del tesseracto una vez ejecutado, se debe tener conocimiento de qué transformaciones se están aplicando en cada parte del programa, si el resultado corresponde con lo esperado según la teoría y qué tan correcta es la representación respecto al modelo algebraico original. El objetivo no es solo hacer que se mueva, sino entender por qué lo hace y qué papel juega el álgebra lineal en cada etapa del proceso.

Este código realizado va a permitir que el proyecto no se limite a un ejercicio de programación, sino que sirva como una aplicación real de los temas vistos en clase, reforzando la comprensión de transformaciones lineales, dimensiones y proyecciones desde un enfoque práctico.

Transformaciones en un Tesseracto de Forma Algebraica

Comenzamos a usar nuestras transformaciones en las funciones de rotación, siendo estas rotaciones lineales que van desde nuestra función “vec4d RotXY” hasta la función “vec4d RotZW”, siendo cada una de estas la rotación de un vector en un plano de cuatro dimensiones.

Como ejemplo tomaremos la función de rotación XY, en la cual tenemos: float

$c = \cos(t)$, $s = \sin(t)$;

$\text{return } \{ p.x * c - p.y * s, p.x * s + p.y * c, p.z, p.w \};$

Si lo vemos como un producto de matrices (correspondiente a las transformaciones), nuestras rotaciones se verían de la siguiente forma:

$$\text{RotXY} = \begin{pmatrix} \cos(t) & \sin(t) & 0 & 0 \\ \sin(t) & \cos(t) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$\text{RotXZ} = \begin{pmatrix} \cos(t) & -\sin(t) & 0 & 0 \\ \sin(t) & \cos(t) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$\text{RotXZ} = \begin{pmatrix} \cos(t) & 0 & \sin(t) & 0 \\ 0 & 1 & 0 & 0 \\ \sin(t) & 0 & \cos(t) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$\text{RotXW} = \begin{pmatrix} \cos(t) & 0 & 0 & -\sin(t) \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \sin(t) & 0 & 0 & \cos(t) \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$\text{RotYZ} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(t) & -\sin(t) & 0 \\ 0 & \sin(t) & \cos(t) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$\text{RotYW} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(t) & 0 & -\sin(t) \\ 0 & 0 & 1 & 0 \\ 0 & \sin(t) & 0 & \cos(t) \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

$$\text{RotZW} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos(t) & -\sin(t) \\ 0 & 0 & \sin(t) & \cos(t) \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}.$$

Con esto definido, podemos declarar las rotaciones correspondientes a cada una de las funciones, esto lo haremos con la siguiente línea de código perteneciente a la función

RotateMATLAB:

```
p = RotXY(p, t * 0.7f);
p = RotZW(p, t * 0.9f);
p = RotXW(p, t * 0.5f);
p = RotYZ(p, t * 0.8f);
```

return p;

Siendo “angulo=tiempo*velocidad” la formula que utilizaremos para rotarla, en este caso estamos utilizando un tiempo de 2 segundos y una velocidad variable entre 0.5 y 0.9, esto para tener una mejor visibilidad de cómo funciona.

Mientras que, para la proyección de 4D a 3D, necesitaremos que nuestras coordenadas estén sobre el eje w, es decir, el eje perteneciente a la cuarta dimensión, esto nos ayudara a obtener la profundidad y poder tener una vista mas cercana o lejana al objeto en nuestro espacio 3D, en este caso lo podemos encontrar en la siguiente línea de código:

```
float w = 4.0f - p.w;

return { p.x / w, p.y / w, p.z / w };
```

En este caso, si w tiene un valor cercano a 4, el punto en el que este se verá más cerca de nosotros, y al contrario, si es menor a 4 se verá más lejos, y nuestra transformación se ve de la siguiente forma:

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1 & 4 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \\ 1 \end{pmatrix}.$$

Luego de esta transformación, cada coordenada se divide entre “4-w”:

$$X=x/4-w \quad Y=y/4-w \quad Z=z/4-w$$

no obstante, las coordenadas de nuestra pantalla comienzan desde la parte inferior izquierda, es decir, que nuestras coordenadas (0,0) se encuentran ahí, lo cual es un problema para que nuestro tesseracto rote correctamente, así que para esto utilizamos:

```
float s = 180.0f;
return {
    int(ScreenWidth() / 2 + p.x * s),
    int(ScreenHeight() / 2 + p.y * s)
};
```

En este caso, estamos moviendo los ejes X y Y, hacia el centro de la pantalla, dividiéndolos entre 2 y sumando el producto de nuestro eje por un escalado en este caso s con valor de 180, por lo que, el punto de origen se mueve al centro. De igual manera esto se puede ver como una transformación.

$$\begin{pmatrix} s & 0 & \frac{w}{2} \\ 0 & -s & \frac{H}{2} \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} =$$

Esto no solo nos ayuda a centralizar las coordenadas, si no que dentro de este mismo espacio de código, ya podemos encontrar nuestra coordenada Z codificada, es decir, que ya tenemos nuestra proyección de 3D a 2D, dándole así profundidad a nuestro trabajo.

Desarrollo y Documentación del Código

1.1 Generación de los vértices en 4D

El tesseracto está compuesto por 16 vértices, definidos por todas las combinaciones de valores posibles en +1 en cada una de sus 4 coordenadas:

1.X

2.Y

3.Z

4.W

En el código realizado, esto se hace mediante un bucle que revisa los bits del número.

```
25
26     for (int i = 0; i < 16; i++) {
27         points4D.push_back({
28             (float)((i & 1) ? 1 : -1),
29             (float)((i & 2) ? 1 : -1),
30             (float)((i & 4) ? 1 : -1),
31             (float)((i & 8) ? 1 : -1)
32         });
33     }
```

Cada bit representa un eje. Esto asegura que el tesseracto conserve su geometría y simetría, sin alterar otros ejes.

1.1 Construcción de las Aristas

Se necesita determinar qué puntos deben conectarse mediante líneas para formar la estructura del hipercubo.

En un hipercubo, cada punto se conecta únicamente con aquellos cuya representación difiere en un solo bit, equivale a modificar una única coordenada manteniendo las demás fijas. De esta manera se garantizan aristas de la misma longitud y una estructura uniforme.

```
vec4d RotateMATLAB(vec4d p, float t) {
    p = RotXY(p, t * 0.7f);
    p = RotZW(p, t * 0.9f);
    p = RotXW(p, t * 0.5f);
    p = RotYZ(p, t * 0.8f);
    return p;
}
```

Cada bit representa un eje. Esto asegura que el tesseracto conserve su geometría y simetría, sin alterar otros ejes.

1.2 Construcción de las Aristas

Se necesita determinar qué puntos deben conectarse mediante líneas para formar la estructura del hipercubo.

En un hipercubo, cada punto se conecta únicamente con aquellos cuya representación difiere en un solo bit, equivale a modificar una única coordenada manteniendo las demás fijas. De esta manera se garantizan aristas de la misma longitud y una estructura uniforme.

```
for (int bit = 0; bit < 4; bit++) {  
    int j = i ^ (1 << bit);  
    if (i < j) edges.push_back({ i, j });  
}
```

- XOR (^) hace que el bit especificado cambie de 0 a 1 o de 1 a 0.
- Esto implica cambiar únicamente una dimensión a la vez.
- El filtro $i < j$ evita duplicar aristas.

El resultado es un conjunto completo de aristas correctamente definidas, que sirve como esqueleto del tesseracto.

1.3 Rotaciones en el espacio de cuatro dimensiones en

un espacio 4D existen seis planos posibles:

XY, XZ, XW

YZ, YW, ZW

Cada uno de estos planos define un movimiento independiente, y el código implementa las seis transformaciones mediante funciones dedicadas que utilizan senos y cosenos, igual que las rotaciones tradicionales en 3D.

Ejemplo en rotación en el plano XW:

```
vec4d RotXW(const vec4d& p, float t) {  
    float c = cos(t), s = sin(t);  
    return { p.x * c - p.w * s, p.y, p.z, p.x * s + p.w * c };  
}
```


Interpretación:

- El eje X y el eje W giran uno respecto del otro.
- Las demás coordenadas (Y y Z) permanecen sin alteración.

```
vec4d RotateMATLAB(vec4d p, float t) {  
    p = RotXY(p, t * 0.7f);  
    p = RotZW(p, t * 0.9f);  
    p = RotXW(p, t * 0.5f);  
    p = RotYZ(p, t * 0.8f);  
    return p;  
}
```

Cada plano posee su propio factor de rotación, lo cual genera un movimiento más complejo, natural y evita errores de que no se logre un buen resultado.

Esta combinación produce un efecto fluido que recuerda al estilo en el que se representan rotaciones en espacios multidimensionales.

1.4 Proyección de 4D a 3D

En la computadora no se puede mostrar un punto con coordenada W directamente. Por ello se realiza una técnica de proyección la cual consiste en “dividir” las coordenadas del espacio tridimensional por un factor relacionado con la dimensión W.

```
vec3d Project4Dto3D(vec4d p) {  
    float w = 4.0f - p.w;  
    return { p.x / w, p.y / w, p.z / w };  
}
```

Interpretación:

- Cuanto mayor sea la magnitud de w, más lejos aparece el punto.
- Simula profundidad adicional proveniente de la cuarta dimensión.
- Es una extensión lógica de la proyección perspectiva tradicional 3D a 2D.

El valor 4.0f actúa como plano de referencia o punto general del sistema.

1.5 Proyección de 3D a 2D

Después de obtener la figura tridimensional, se necesita adaptar esa proyección a la ventana gráfica bidimensional. Esto se consigue escalando cada punto y trasladándolo al centro de la pantalla.

```
vec3d Project4Dto3D(vec4d p) {  
    float w = 4.0f - p.w;  
    return { p.x / w, p.y / w, p.z / w };  
}
```

Realizamos dos operaciones fundamentales:

1. Escalamiento: amplifica la figura para que sea visible.
2. Traslación: mueve el origen (0,0) al centro de la pantalla.

Esto nos ayuda a completar el proceso de reducción dimensional de 4D a 3D y posteriormente a 2D. Cada etapa conserva la mayor cantidad de relaciones geométricas posibles, sin distorsionar la estructura global del tesseracto.

1.6 Renderizado final

Ya proyectadas las posiciones en la pantalla, el programa procede a dibujar las líneas entre los pares de puntos definidos en edges.

```
for (auto& e : edges) {  
    auto a = Project3Dto2D(proj[e.a]);  
    auto b = Project3Dto2D(proj[e.b]);  
    DrawLine(a, b, olc::CYAN);  
}
```

El color cyan lo seleccionamos para que el modelo resalte sobre el fondo negro, y para facilitar la identificación visual de intersecciones al generarse los giros del hiper cubo.

Adicionalmente, en cada ciclo se incrementa el tiempo acumulado.

```
totalTime += fElapsedTime;
```

Esto asegura que las rotaciones sean continuas, suaves y dependientes del tiempo real que transcurre entre cuadros. Cada cuadro muestra el tesseracto en una posición ligeramente distinta debido a la suma continua del tiempo en las funciones de rotación.

La estructura del tesseracto se mantiene por la relación entre sus puntos, las rotaciones no alteran su forma fundamental y las proyecciones permiten que un objeto que existe matemáticamente en cuatro dimensiones pueda ser observado en una pantalla aplicando temas como los que vemos en Algebra Lineal.

El resultado es una visualización si tratáramos de verlo en 4D, sí representa cómo se comportaría un hipercubo si pudiera ser observado desde un entorno tridimensional.

1.7 Optimización y estabilidad visual de la animación

Además de las rotaciones y proyecciones, el programa también incorpora ciertos ajustes para asegurar que la animación del tesseracto sea estable y se mantenga fluida mientras está en movimiento. Aunque este aspecto no se ve a simple vista en el dibujo final, forma parte del funcionamiento interno del código y es necesario para que las transformaciones se apliquen con continuidad y sin saltos bruscos.

El primer elemento que interviene es la variable `totalTime`, que se actualiza cada vez que se ejecuta el método `OnUserUpdate`. Con esto se evita que las rotaciones dependan de la velocidad del equipo o del rendimiento del programa. Al usar el tiempo acumulado como parámetro, las rotaciones avanzan de forma pareja y el movimiento del tesseracto no se acelera ni se detiene de manera irregular. En pocas palabras, la animación se vuelve estable y predecible, sin importar cuántos cuadros por segundo esté generando la computadora.

Otro punto que contribuye a la estabilidad es la decisión de calcular todas las transformaciones antes de dibujar cualquier línea. El programa primero procesa los 16 puntos, les aplica las rotaciones y luego realiza las proyecciones. Solo cuando todos los puntos están actualizados, se comienza a dibujar cada arista. Esto evita errores en los que algunas partes del tesseracto podrían aparecer desplazadas respecto de otras.

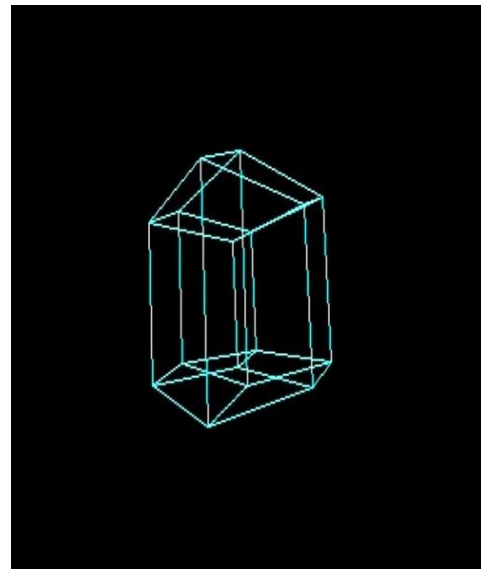
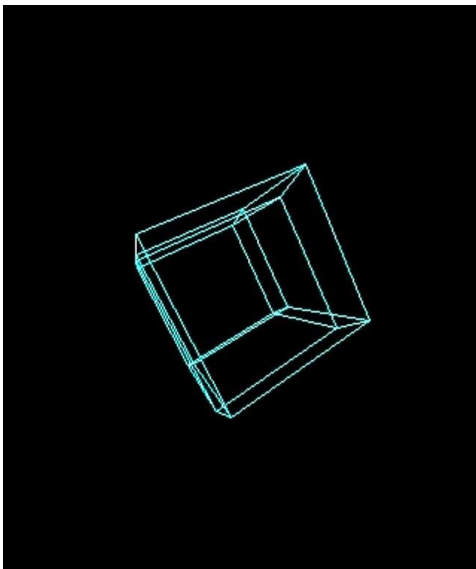
Gracias a estos ajustes, el tesseracto puede apreciarse con claridad, conservando su estructura en cada rotación y permitiendo analizar de manera visual cómo actúan las transformaciones lineales en un espacio de cuatro dimensiones.

3.Resultados

Al ejecutar dicho programa en Visual Studio, se produce una representación dinámica y estable de un tesseracto rotando en un espacio 4D. Los puntos siguen trayectorias coherentes que reflejan con precisión la interacción entre dimensiones, y las aristas mantienen su proporción en todo momento.

Al concluir dicho programa, algunos de los objetivos obtenidos se encuentran.

- Implementación de rotaciones en 4D.
- Visualización del hipercubo de manera correcta sin errores.
- Proyecciones consistentes que conservan relaciones geométricas
- El hiper cubo – tesseracto no tuvieron deformaciones no esperadas.
- La ejecución fue legible pese a la complejidad del movimiento generado



La visualización obtenida confirma que las transformaciones aplicadas funcionan de manera correcta y estable. El tesseracto mantiene su estructura a lo largo de toda la animación, las aristas no presentan saltos bruscos y las rotaciones sobre distintos pares de ejes producen deformaciones esperadas para un objeto que no existe físicamente en nuestro espacio tridimensional. La transición entre la proyección 4D a 3D y luego 3D a 2D no genera errores. De igual manera se mostrará un link donde vendrá video de cómo es que se ejecuta dicho cubo en un espacio 4D.

Los resultados no solo muestran que el programa cumple con lo planteado, sino que también permiten apreciar de forma visual el efecto real de las transformaciones lineales

Conclusión

Al finalizar este proyecto, concluimos que los temas que abordamos en álgebra lineal solo alcanzan su verdadero sentido cuando se aplican en un contexto práctico. Trabajar con un tesseracto permitió comprobar que los vectores, matrices y transformaciones no son herramientas aisladas, sino piezas que se utilizan para describir y manipular objetos en distintos espacios. Al representar un modelo en cuatro dimensiones, fue necesario apoyarse en cada uno de los temas vistos en la materia.

El proceso de implementar las rotaciones en planos del espacio 4D como XY, XW, YZ y ZW, da explicación de cómo se forman y aplican las matrices correspondientes, ya que cada una define un movimiento específico que afecta solo a ciertos componentes del vector. Esto permitió distinguir con claridad lo que representa una transformación lineal, una regla que, sin importar la dimensión, mantiene una estructura algebraica y un comportamiento. Al verificar la animación final, es evidente que la figura está justamente funcionando como se quería, dando explicación a muchos de los temas que se abordaron durante el semestre.

La parte de las proyecciones también aportó una comprensión más profunda sobre cómo se reduce dimensionalidad sin perder la coherencia del objeto. Primero, la proyección de 4D a 3D dependió directamente de la componente w , que actuó como un factor de profundidad, y posteriormente, la proyección de 3D a 2D nos permitió trasladar la figura a la pantalla. Este proceso mostró cómo una transformación puede adaptarse a un objetivo específico, en este caso, convertir un modelo de álgebra en una representación visual entendible.

Desde el punto de vista técnico, el uso de C++ y la librería `olcPixelGameEngine` demostró que la programación y el álgebra lineal se complementan de manera natural. El código no solo sirvió para dibujar líneas; permitió automatizar cálculos que manualmente serían imposibles de visualizar. Cada arista del tesseracto, cada movimiento y cada cambio de perspectiva es resultado directo de operaciones matriciales ejecutadas de forma continua.

Este proyecto deja ver que el álgebra lineal no se limita a resolver sistemas, calcular determinantes o manipular vectores, sino que es una herramienta con la capacidad de describir otros temas que van más allá de nuestra intuición. Ver en pantalla cómo se comporta un objeto de cuatro dimensiones hace que el álgebra cobre un sentido distinto.

